A pressure control device for force application during optical coherence tomography imaging and control mechanism for low-level laser therapy

---

Brady Perkins, December 2025

Biodesign Lab, Asia University, Taichung City, Taiwan

1. Pressure control and recording device

Overview

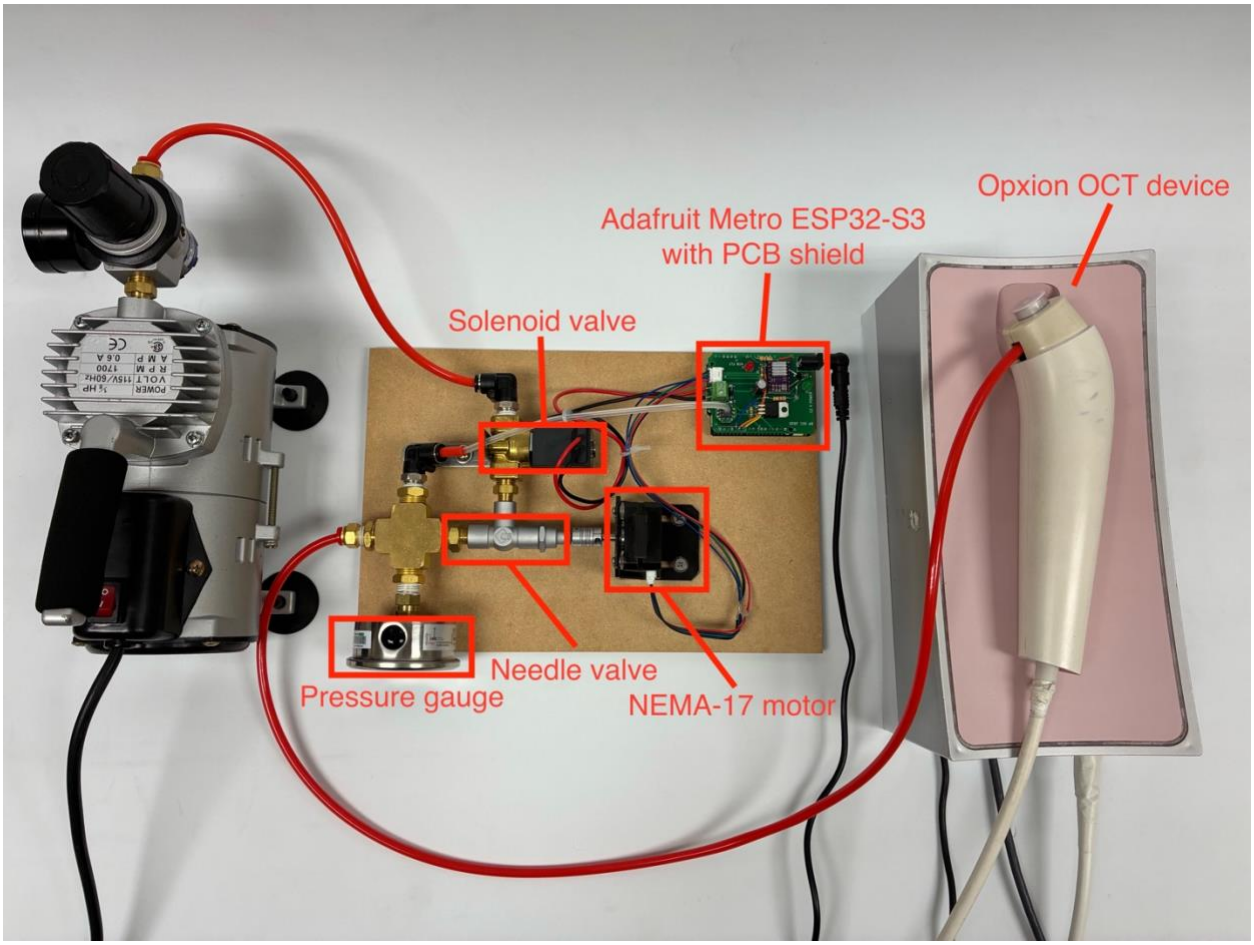

Figure 1: A labeled overhead view of the OCT pressure-control device.

A device has been created which uses a NEMA-17 stepper motor to control a needle valve used to adjust the airflow fed into an optical coherence tomography imaging device, as shown in Figure 1. A solenoid valve is also used to turn airflow on and off for certain air-burst applications.

Device control is accomplished using an Arduino IDE-compatible ESP32-S3-based microcontroller, the Adafruit Metro ESP32-S3. A custom shield PCB with Arduino UNO-compatible dimensions was also manufactured according to the schematic shown in Figure 2.

Power delivery PCB

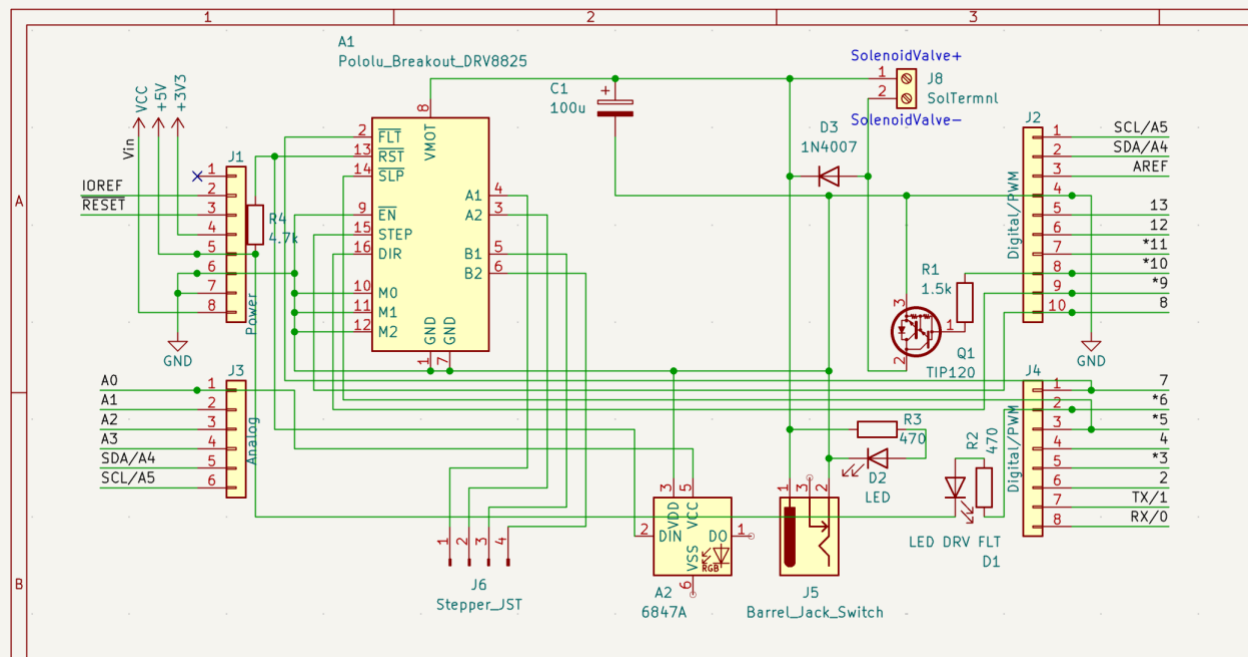

Figure 2: The schematic for the shield PCB.

The shield PCB is a simple two-layer design which primarily enables power delivery to the two main high-power components of the device: the solenoid valve and stepper motor. The board houses a Pololu DRV8825-compatible stepper motor driver (Pololu Corporation, Las Vegas, Nevada, United States) (labeled A1 in the Figure 2 schematic), in addition to a TIP120 power transistor, used to power the stepper motor and solenoid valve respectively. In addition, a CFSensor 6847A analog electronic pressure sensor module with detectable pressure range 0 – 100 kPa (CFSensor, Wuhu City, Anhui Province, P.R.C.) (labeled A2 in the Figure 2 schematic) is housed and connected to the microcontroller using the PCB.

The PCB is assembled as shown in the two- and three-dimensional CAD models in Figure 3 and Figure 4.

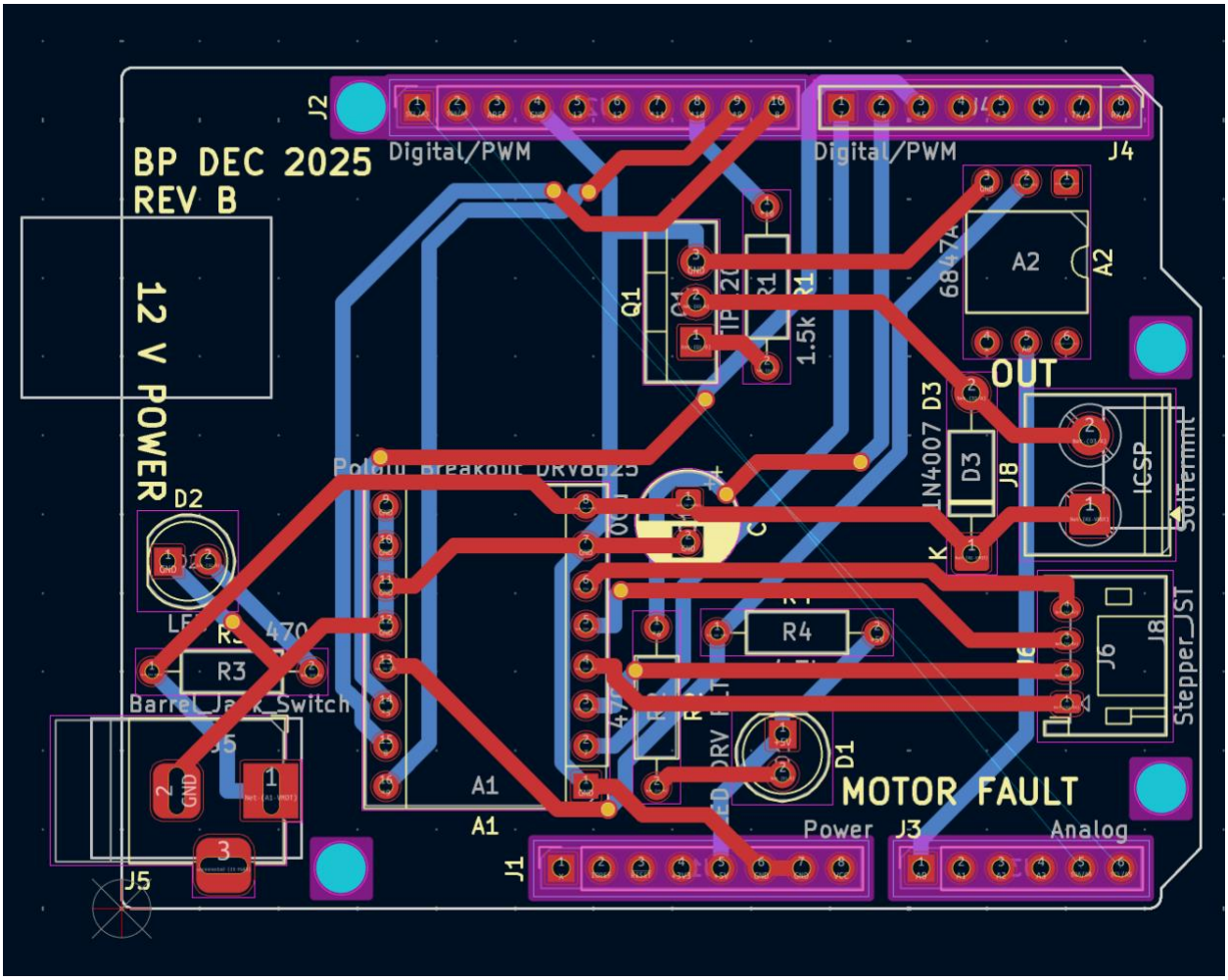

Figure 3: The two-dimensional KiCad model of the shield PCB (red traces lie on front layer, blue traces on back layer).

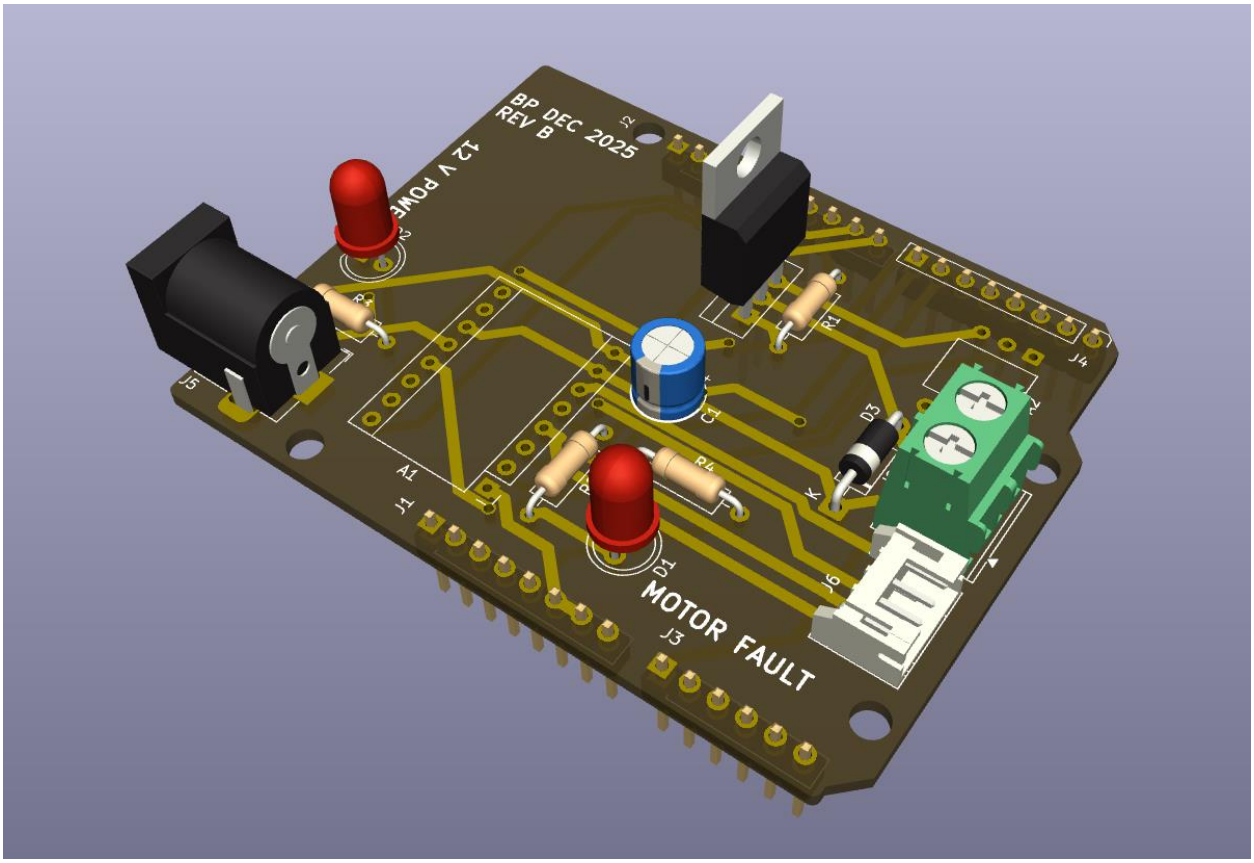

Figure 4: A three-dimensional KiCad model of the board semi-populated.

The solenoid and stepper motor are each powered using a 12 V DC power supply, which is fed to the PCB via the 2.1 mm barrel connector shown in the 3D model in Figure 4. An LED is used to visually verify 12 V power connection, and the 12 V power is fed directly to the VMOT and GND pins on the top-right of the DRV8825 (as shown in the pinout diagram in Figure 5) and to the solenoid through the TIP120 power transistor, which connects and disconnects the GND signal of the 12 V connection fed to the screw terminal connector on the right side of the board (to which the solenoid valve is attached).

The diode placed just before the screw terminal is a 1N4007 power diode to create a low-resistance path for flyback current (current of opposite polarity to the current supplied by the 12 V DC input) generated by the collapsing magnetic field of the inductor in the solenoid valve upon its power-off.

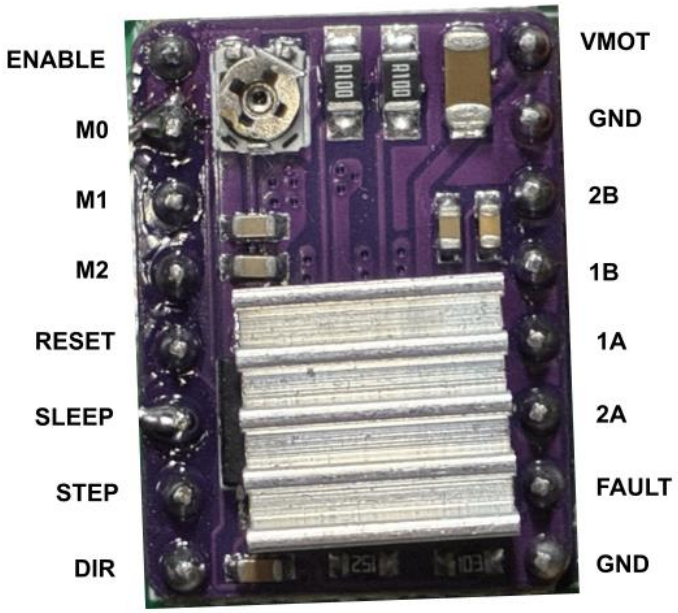

Figure 5: The pinout of the Pololu-style DRV8825 stepper motor driver.

Other connections also shown in Figure 5 include the motor coil connections: 2B, 1B, 1A, and 2A, as well as the "FAULT" signal pin, which is activated any time the H-bridge FETs driving current to the motor coil connections are disabled, a logic-level GND pin, ENABLE, RESET, SLEEP, STEP, DIR, and the (here unused) fractional stepping pins, M0, M1, and M2.

Signals of the DRV8825 that this pressure control device utilizes include the necessary STEP and DIR – which give instruction for the motor to step when pulsed and change the motor direction based on high or low input, respectively – as well as the SLEEP pin used to disable the H-bridge FETs when the motor does not need to receive current (while it is not moving), and the FAULT pin, which is used to read whether the H-bridge FETs have been disabled while the motor should be turning. All other pins – ENABLE, M0, M1, M2, and logic GND – are connected to ground.

The VMOT and GND motor power inputs (12 V DC) also have a 100 µF capacitor placed across them in order to ensure a clean and constant motor power supply input to the driver.

Device and microcontroller connections

Microcontroller general-purpose input/output (GPIO) pins are used to communicate between the devices housed on the PCB and the microcontroller which sits underneath the PCB. The pin connections of each device are as given in Table 1.

Table 1: GPIO-device connections.

Arduino UNO-compatible GPIO pin	Device connection on PCB
5	DRV8825 SLEEP
6	MOTOR FAULT LED
7	DRV8825 FAULT
8	DRV8825 STEP
9	DRV8825 DIR
10	Solenoid (TIP120 base)
A0	Pressure sensor output (6847A pin 5)

Pins 11-13 remain unused to avoid interference with the SPI interface on many Arduino UNO-compatible microcontroller designs. All ground connections on the Arduino are used as ground connections on the shield PCB, and the highest positive-voltage output (the voltage output pin closest to ground in the power row of Arduino UNO-compatible designs) is used as the logic-level voltage connection (+5 V on Arduino UNO and +3.3 V on ESP32 platforms) for the DRV8825 and 6847A pressure sensor.

The Adafruit Metro ESP32-S3 microcontroller was purchased from Digi-Key Taiwan (Digi-Key Electronics, Thief River Falls, Minnesota, United States), while initial PCB prototypes as shown in Figure 1 were manufactured by Yuan Guang PCB-togo (Yuan Guang PCB-togo Electronics, Inc., Taoyuan City, Taiwan) and later PCB units based on the schematic shown in Figure 3 manufactured by PCBway (Shenzhen JDB Technology Co., Ltd., Shenzhen City, Guangdong Province, P.R.C.). Electronics components were purchased at Jin Hua Electronics (今華電子, Taichung City, Taiwan).

Airflow within the device

The path of the air through the device begins at the air intake, which is connected to a positive-pressure air compressor through a pressure regulator of a low pressure (<100 kPa) set by the user. The air first travels through the solenoid valve, then through the needle valve to a three-way splitter that acts as a pressurized chamber connected to both the 6847A electronic analog pressure sensor module, a mechanical pressure gauge, and the output tube to be connected to the OCT imaging device.

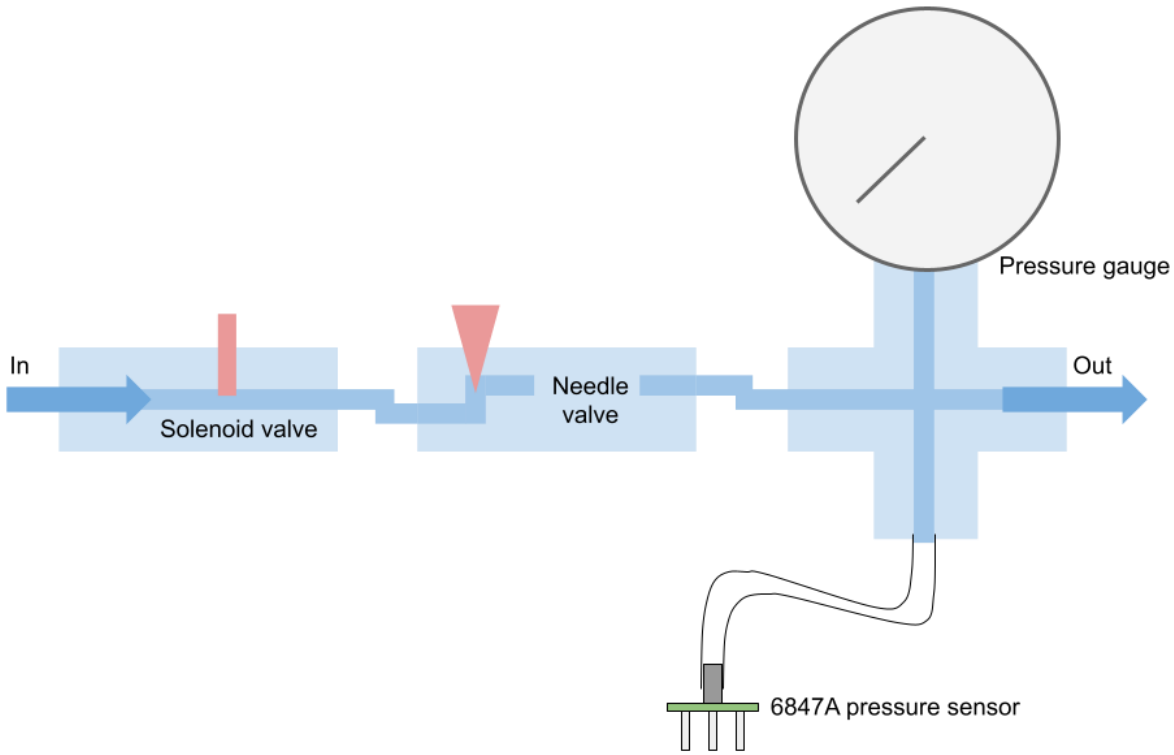

Figure 6: A map of the airflow through the device.

The flow of air through the device is given in a simple map in Figure 6, with airflow control joints represented in red and arrows representing the direction of flow. The solenoid valve has two states: fully open and fully shut, while the needle valve uses a varying-width needle to adjust the airflow with precision.

Airflow control mechanical connections

The stepper motor and needle valve are connected using an axle coupler of internal diameter 5 mm. The diameter of the motor axle is 5 mm exactly, and the diameter of the needle valve axle is 4.5 mm. The additional volume between the needle valve axle and coupler are filled using solder.

The connections between each of the airflow devices are measured in United States imperial units (inches), while the tubes are measured in metric units (millimeters).

The internal diameter of the screw connections on the solenoid valve are 0.375 inches, the screw connections on the needle valve are 0.25 inches, and screw connections on the three-way splitter are 0.5 inches. Screw size adapters are used to interface each component.

The screw diameter of the tube connections to the input, output, and pressure sensor are each 0.375 inches, and the diameter of the connected tubes are 4 mm with a wall thickness of 1 mm (making for a total outer tube diameter of 6 mm).

The smaller tube connection to the pressure sensor has an outer diameter of 4 mm with a wall thickness of 1 mm, for an inner diameter of 2 mm.

The screw diameter of the pressure gauge connected to the three-way splitter is 0.5 inches.

Each of the hardware components were purchased at ALD Hardware (振宇五金, Taichung City, Taiwan), except for the needle valve and solenoid valve, both purchased at Wen Jun Hardware (文俊五金, Taichung City, Taiwan).

Before the air input is a pressure regulator connected to the compressor, model Mindman MAR 300-8A (Mindman Pneumatics, Taipei City, Taiwan).

Data input and output

Data input and output is accomplished using the microSD card slot on the Adafruit Metro ESP32-S3 board.

On startup, the microcontroller reads from two files in a folder on the SD card called “sys”: calibration.csv and wifi.csv.

The file calibration.csv contains the values of the average of three zero-points of the 6847A pressure sensor, used as the reference for 0 kPa, as well as the average value of three high calibration points taken at 20 kPa. The values are separated by a comma, in this format: 500,1000. Values are read from the pressure sensor's output pin via the Arduino analogRead function, so in this example, the reference for 0 kPa is analogRead result value of 500, while the reference for 200 kPa is an analogRead result value of 1000.

The file wifi.csv contains the authentication information for a wireless network, used so that the Arduino can retrieve the date & time, used for dating trial data when it is saved to an output file on the SD card.

While running, two additional folders on the SD card are used for input and output of information. These are "in" and "out". The "in" folder contains CSV files with two columns: time, in milliseconds, on the left and motor position, in degrees, on the right. These are treated as programs which can be loaded on the microcontroller. The "out" folder is the folder which dated CSV output data files are placed in after a trial is run. These CSV files contain four columns, in order, time in milliseconds, output force in grams, tube pressure in kilopascals (kPa), and motor position in steps, where one step is equal to 1.8 degrees of rotation.

Names for these files are generated using the date retrieved from the Internet in the format YYYYMMDD_HHmmSS_oct.csv, and the initial date used is 2025-01-01 00:00:00 if unable to retrieve a time from the Internet (for example, if the network is not connected to the Internet or if the microcontroller is unable to connect to the network).

Communication between the user and the microcontroller is accomplished over a serial terminal at 115200 baud, connected to a computer via a USB cable.

Device calibration

Device calibration is accomplished over the serial connection between the ESP32 microcontroller and a computer. A calibration process is begun by entering "cal" through the terminal and following on-screen prompts to take three pressure sensor measurements for a high calibration point and zero calibration point. Pressure values read by the sensor are not absolute, but instead relative to atmospheric pressure.

The high calibration point is set to a pressure of 20 kPa, and the user is asked to adjust the position of their finger on the end of the outlet tube until the pressure output reads 20 kPa, then to enter the word “next” into the serial terminal. After the measurement of three calibration 20 kPa calibration points, the user is asked to remove their finger from the end of the outlet tube so that the device can take three zero points at atmospheric pressure (0 kPa).

Software control

Input to the device is given via an input CSV file to be interpreted by the software on the microcontroller. The CSV file contains two columns, with each row representing a point in the movement of the motor: a timestamp (in milliseconds) in the left column, and a motor position (in degrees) in the right column. An example is given in Table 2.

Table 2: An example CSV with motor program input data.

0	0
500	0
1500	300
2000	300
3000	0
4000	0

Upon serial connection, the user types “load”, followed by the name of the file as it is listed in the SD card’s /in/ folder, without file extension: for example, for the file /in/program.csv, the user enters “load program”. When a program is loaded, the CSV is converted to a series of instructions to be processed by a program interpreter, which specify the amount of time to record for (in 25 Hz frames to match the recording rate of the Opxion OCT device), the amount of time to keep the solenoid open for (in milliseconds), followed by pause, motor velocity, and motor position instructions. The corresponding program to the example in Table 2 is:

r100,s4000,t500,v300,m300,t500,m0,t1000

This program specifies to record a CSV file for 100 frames (4 seconds), to open the solenoid for 4000 milliseconds (so that the solenoid is open for the same time period as the recording), followed by a 0.5-second pause timer before the motor velocity is set to 300 degrees per second

and moved to position 300 degrees. Another 0.5-second pause timer precedes an instruction to move the motor back to position 0 degrees and to wait a final period of 1 second.

To run the program, the user enters “run” into the serial terminal.

The initial recording and solenoid time are calculated by the CSV interpreter based on the period from the first timestamp listed in the CSV file to the final timestamp. The maximum allowed motor speed is 400 degrees per second.

Upon running of the program, a program interpreter reads these serial instructions and moves the motor and solenoid accordingly. Recording and solenoid opening instructions are non-blocking, so that recording and solenoid open time can occur alongside motor movement. The timer instruction is blocking, to allow for periods during the recording where the motor should not move, for example, to generate buffer time at the beginning of the program to allow for delay before the start of the OCT recording.

The OCT recording and motor programs are begun asynchronously, since the OCT device relies on control software shipped by its manufacturer which does not allow for remote instruction from other running programs. CSV line recording is done at 10 Hz, while OCT frame recording is done at 25 Hz.

Programs in the comma-separated serial format can also be entered directly after the “run” command. For example, if the solenoid must be turned on and off while the needle valve is open, this cannot be accomplished using the CSV interpreter. A program must be entered directly after “run”, for example:

r100,s500,t1000,s500,t1000,s500,t1000,s500,t1000

This example program records for 100 frames and opens the solenoid for 0.5 seconds over four 1-second time intervals, causing the solenoid to toggle on and off every 0.5 seconds.

Upon completion of a record instruction, the data is saved to the SD card’s /out/ folder with its dated filename and four columns containing time, output force in grams, tube pressure in kPa, and motor position in steps (to avoid using the compute necessary for float math to convert motor position in steps to motor position in degrees during data output).

Output force in grams is calculated using a simplified form of the Bernoulli equation which assumes a horizontal tube network (no change in gravitational potential) and a tissue sample less than one tube diameter (less than 4 mm) away from the end of the outlet. This is given in Equation (1).

$$F = 2A\Delta P \tag{1}$$

Where A is equal to the cross-sectional area of the outlet tube in square meters, and ΔP is equal to the pressure difference between the inner tube network and the outer atmosphere.

This simplified relationship is derived from the Bernoulli equation:

$$F = P_{atm} \cdot (A) + \rho Q V_{out} \tag{2}$$

In Equation (2), V_{flow} is equal to the velocity of the air leaving the tube, and ρ is equal to the density of the fluid (air). Q is the volumetric flow rate of the fluid, equal to the cross-sectional area of the tube times the velocity of the fluid.

$$Q = AV_{out} \tag{3}$$

Substituting Equation (3) into Equation (2) gives Equation (4):

$$F = P_{atm}A + \rho A V_{out}^2$$

$$F = A(P_{atm} + \rho V_{out}^2) \tag{4}$$

With a horizontal network of tubing, Equation (5) is assumed:

$$P_{in} + \frac{1}{2}\rho V_{in}^2 \approx P_{out} + \frac{1}{2}\rho V_{out}^2 \tag{5}$$

If the input velocity is very low (outlet of air pressure regulator and tubing network inlet are at about the same pressure, so the pressurized tube can be approximated as a large air reservoir):

$$\Delta P = P_{in} - P_{atm} \approx \frac{1}{2}\rho V_{out}^2$$

$$\rho V_{out}^2 = 2\Delta P$$

(6)

Substituting Equation (6) back into Equation (4) gives Equation (7):

$$F = A(P_{atm} + 2\Delta P)$$

(7)

Since the pressure at the outlet, P_{atm}, is atmospheric pressure, P_{atm} can be approximated as insignificant compared to ΔP (the pressure inside the tubing network).

This leaves Equation (1):

$$F = A(2\Delta P)$$

$$F = 2A\Delta P$$

(1)

In Equation (8), A is in units of square meters, and ΔP is in units of kilopascals (kPa). The output force in this equation is in units of Newtons, which are divided by a factor of 1000/9.81 for conversion from Newtons to grams. A is calculated as πr^2, where r = 0.002 meters (2 millimeters, half of the tube's 4 mm diameter), for a constant result of 0.00001257 m^2. ΔP is calculated by scaling the voltage reading output value from analogRead to 0 kPa and 20 kPa calibration points using Equation (9).

$$\Delta P = \frac{v}{k} \cdot 20$$

(8)

In Equation (9), v is the result from analogRead (the reading from the 6847A pressure sensor through a microcontroller analog input pin), and k is the difference between the analogRead

values of the high calibration point (at 20 kPa) and the zero calibration point, while 20 is the 20 kPa scaling factor.

A note about the choice of microcontroller: the software for the pressure control device addresses each function asynchronously from inside the main loop in order to enable multitasking on any microcontroller. However, in addition to wireless Internet and removable storage capability, the Adafruit Metro ESP32-S3's additional speed compared to an Arduino UNO helps to prevent slowdown, aiding motor movement accuracy during output CSV writing and force calculations.

Low-level laser therapy control device

Overview

The control device for low-level laser therapy application reuses the PCB design from the pressure control device, making use of the 12 V DC output originally for the solenoid valve for the laser power supply. The device setup is given in Figure 7.

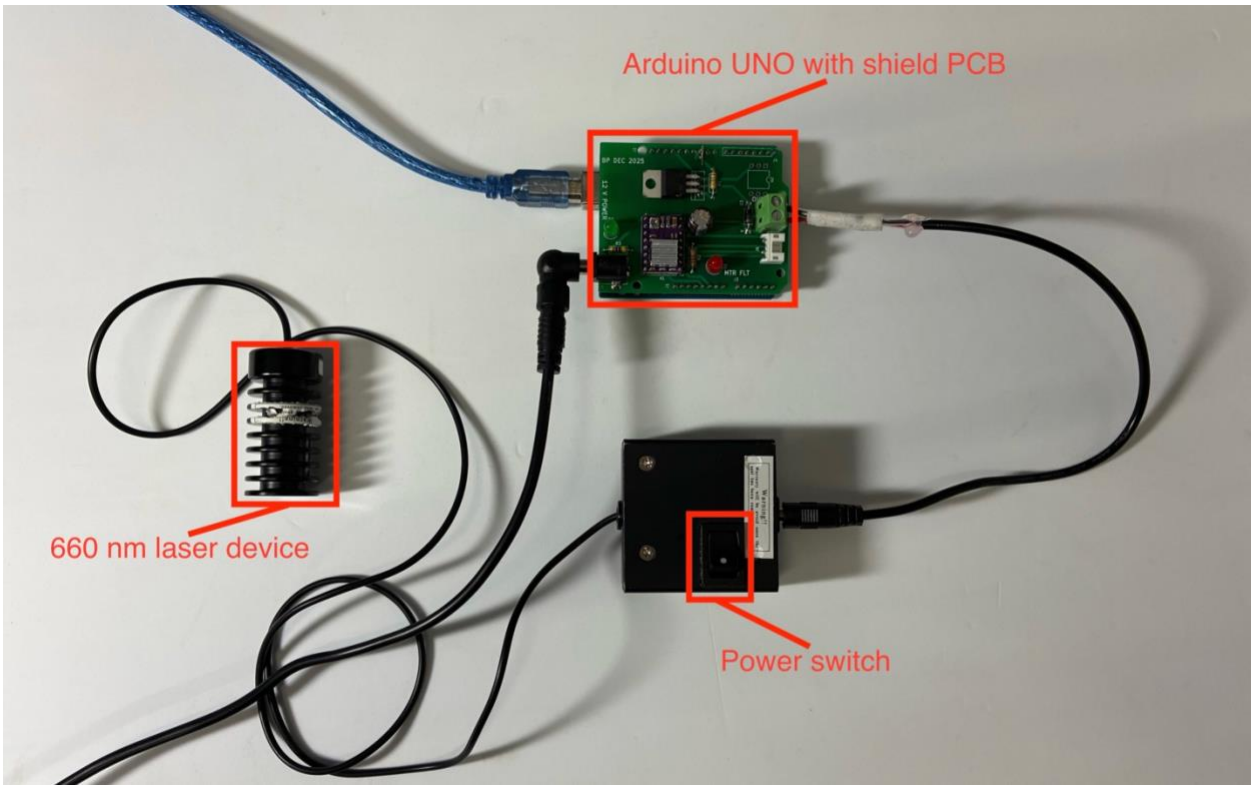

Figure 7: Low-level laser therapy control device.

The laser used is a 660 nm-wavelength device with a power input box that uses a DC voltage regulator to convert a range of higher-voltage input power between 8 and 30 V into the 3.3 V power used by the laser device, as well as containing a power switch used to turn the device on and off.

The microcontroller utilized in this device is an Arduino UNO instead of the Adafruit Metro ESP32-S3 board used for the pressure control device, since removable storage, Internet capability, and additional speed are not necessary.

Laser control/user interface

Laser control is also done via a serial terminal, running at 9600 baud on the Arduino Uno instead of the 115200 baud utilized by the ESP32 microcontroller. The user can only enter commands into the serial terminal in one format:

nn,dd

where *nn* is the desired flashing rate of the laser (in Hz), and *dd* is the desired duty cycle of the laser refresh (in percent). The default setting is:

1,50

which corresponds to a refresh rate of 1 Hz and a 50 percent duty cycle.

Trials are conducted by deciding the amount of time that the laser should remain on for, and this time is tracked using an external stopwatch which is begun as the power switch on the laser box as shown in Figure 7 is turned on, and where the power switch is turned back off once the stopwatch reaches the set amount of time.

Resources

Pressure control device (GitHub):

https://github.com/bap10357/llt-controller

Low-level laser therapy device program (GitHub):

https://github.com/bap10357/oct-pressure-device